数据处理1

Learning outcomes

By the end of this session, you should be able to:

- 数据清洗

缺失值

缺失值:

- NaN: not a number
- 用NaN代替缺失值

```python
import pandas as pd
import numpy as np
df = pd.DataFrame({'rating_A': [np.nan, 3, 4,np.nan , 3, np.nan ],
'rating_B': [np.nan, 3, 4,5, 3, 4]},
index=['A001', 'A002', 'A003','A004', 'A005', 'A006'])
df
```

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

缺失值

如何识别缺失值：

- Isnull函数
- 含有缺失值的列：df.isnull().any()
- 含有缺失值的列总和：df.isnull().any().sum()

```
# isnull 函数
df.isnull().any()#含有缺失值的列
```

✓ 0.6s

```
rating_A    True
rating_B    True
dtype: bool
```

```
df.isnull().any().sum()#含有缺失值的列总和
```

✓ 0.1s

```
2
```

缺失值

如何识别缺失值：

- 含有缺失值的行：

  df.isnull().any(axis=1)

- 含有缺失值的行总和：

  df.isnull().any(axis=1).sum()

```
df.isnull().any(axis=1) #含有缺失值的行
```

✓ 0.4s

```
A001     True
A002    False
A003    False
A004     True
A005    False
A006     True
dtype: bool
```

```
df.isnull().any(axis=1).sum() #含有缺失值的行总和
```

✓ 0.6s

```
3
```

缺失值

如何识别缺失值：

- 全部为空的行：

  df.isnull().any(axis=1)

- 全部为空的行总和：

  df.isnull().any(axis=1).sum()

```
df.isnull().all(axis=1) #全部为空的行
```

✓ 0.7s

```
A001     True
A002    False
A003    False
A004    False
A005    False
A006    False
dtype: bool
```

```
df.isnull().all(axis=1).sum() #全部为空的行总和
```

✓ 0.4s

```
1
```

处理缺失值

如何处理缺失值:

1.删除:

- df.dropna()
- 默认对行进行操作
- axis=0: 对行进行操作
- axis=1:对列进行操作

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

```
na_default=df.dropna()
na_default
```

✓ 0.6s

	rating_A	rating_B
A002	3.0	3.0
A003	4.0	4.0
A005	3.0	3.0

处理缺失值

如何处理缺失值：

1.删除：

- df.dropna()
- axis=1:对列进行操作

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

```
# 对列进行删除
na_col=df.dropna(axis=1)
na_col
```

✓ 0.6s

A001
A002
A003
A004
A005
A006

处理缺失值

如何处理缺失值：

1.删除：

- df.dropna()
- 全部为空才删除

  df.dropna(how='all')

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

```
# 全部为空才删除
na_1=df.dropna(how='all')
na_1
```

✓ 0.7s

	rating_A	rating_B
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

处理缺失值

如何处理缺失值：

1.保留一定数量的观测值：

- df.dropna()
- thresh=1

(只要其中一列有值就被保留)

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

```
# 保留一定数量的观测值
na_2=df.dropna(thresh=1)
na_2
```

✓ 0.6s

	rating_A	rating_B
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

Practice 1

```python
import pandas as pd
import numpy as np
df = pd.DataFrame({'rating_A': [np.nan, 3, 4,np.nan , 3, np.nan ],
'rating_B': [np.nan, 3, 4,5, 3, 4]},
index=['A001', 'A002', 'A003','A004', 'A005', 'A006'])
df
```

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

用dropna()函数对上述数列中的缺失值进行删除操作练习

处理缺失值

如何处理缺失值：

2.填充：

- 方法一：fillna()函数
- 方法二：replace（）函数
- 方法三：sklearn数据处理方法

处理缺失值

方法一：fillna()函数填充

缺失值填充

Fillna()函数进行填充：

- 用前面的‘pad’值进行填充
- df.fillna(method=‘pad’)

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

```
na_fill_2=df.fillna(method='pad') # 用后面的‘backfill’或者前面的'pad'值进行填充
na_fill_2
```

✓ 0.6s

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	4.0	5.0
A005	3.0	3.0
A006	3.0	4.0

缺失值填充

Fillna()函数进行填充：

- 用后面的‘backfill’值进行填充
- df.fillna(method=‘backfill’)

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

```
na_fill_2=df.fillna(method='backfill') # 用后面的‘backfill’或者前面的'pad'值进行填充
na_fill_2
```

✓ 0.4s

	rating_A	rating_B
A001	3.0	3.0
A002	3.0	3.0
A003	4.0	4.0
A004	3.0	5.0
A005	3.0	3.0
A006	NaN	4.0

缺失值填充

Fillna()函数进行填充：

- 用平均值进行填充
- df.fillna（df.median()）

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

```
na_fill_3=df.fillna(df.median()) # 平均值去填充
na_fill_3
```

✓ 0.3s

	rating_A	rating_B
A001	3.0	4.0
A002	3.0	3.0
A003	4.0	4.0
A004	3.0	5.0
A005	3.0	3.0
A006	3.0	4.0

缺失值填充

Fillna()函数进行填充：

- 用0进行填充
- df.fillna（0）

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

```
na_fill_1=df.fillna(0)
na_fill_1
```

✓ 0.4s

	rating_A	rating_B
A001	0.0	0.0
A002	3.0	3.0
A003	4.0	4.0
A004	0.0	5.0
A005	3.0	3.0
A006	0.0	4.0

处理缺失值

- 方法二：replace（）函数填充

缺失值填充

replace()函数进行填充：

replace(np.nan,0)

	rating_A	rating_B
A001	NaN	NaN
A002	3.0	3.0
A003	4.0	4.0
A004	NaN	5.0
A005	3.0	3.0
A006	NaN	4.0

```python
# 方法二：replace
na_rep=df.replace(np.nan,0)
na_rep
```

	rating_A	rating_B
A001	0.0	0.0
A002	3.0	3.0
A003	4.0	4.0
A004	0.0	5.0
A005	3.0	3.0
A006	0.0	4.0

处理缺失值

- 方法三： sklearn数据处理方法

Sklearn数据处理方法

Sklearn数据处理：

- 先导入包
- 再制定填充策略（平均数，中位数，众数）
- 数据缺失值模型

```python
# 方法三：sklearn 数据预处理方法
from sklearn.impute import SimpleImputer
nan_model=SimpleImputer(missing_values=np.nan,strategy='median')
# strategy :mean、median、most_frequent
na_fill_model=nan_model.fit_transform(df)
pd.DataFrame(na_fill_model)
```

✓ 1.2s

	0	1
0	3.0	4.0
1	3.0	3.0
2	4.0	4.0
3	3.0	5.0
4	3.0	3.0
5	3.0	4.0

Sklearn数据处理方法

Sklearn数据处理：

- 数据缺失值模型
- 如何转换为data frame

```python
# 方法三: sklearn 数据预处理方法
from sklearn.impute import SimpleImputer
nan_model=SimpleImputer(missing_values=np.nan,strategy='median')
# strategy :mean、median、most_frequent
na_fill_model=nan_model.fit_transform(df)
pd.DataFrame(na_fill_model
             ,columns=['rating_A','rating_B']
             ,index=['A001', 'A002', 'A003','A004', 'A005', 'A006'])
```

✓ 0.4s

	rating_A	rating_B
A001	3.0	4.0
A002	3.0	3.0
A003	4.0	4.0
A004	3.0	5.0
A005	3.0	3.0
A006	3.0	4.0

Practice 2

1.用以下三种方法进行数据填充：

- 方法一：fillna()函数
- 方法二：replace（）函数
- 方法三：sklearn数据处理方法